

TÜRKİYE CUMHURİYETİ
YILDIZ TEKNİK ÜNİVERSİTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ



OTONOM AJAN YAKLAŞIMIYLA KOD VERİ
TABANINDAN EĞİTİM İÇERİĞİ ÜRETEN YAPAY ZEKA
TABANLI YAZILIMIN TASARIMI VE GELİŞTİRMESİ

21011035 – BORA İLCİ
21011506 – KAAN EMRE KARA

BİLGİSAYAR PROJESİ

Danışman
Prof. Dr. Mehmet Sıddık AKTAŞ

Kasım, 2025

TEŞEKKÜR

Yıldız Teknik Üniversitesi, tarihi 1911'e dayanan Türkiye'nin en eski 3. üniversitesi olmasının yanı sıra İstanbul'da bulunan yedi devlet üniversitesinden biridir. Aynı zamanda ülkenin en iyi üniversitelerinden biri olarak kabul edilmektedir. Üniversitemizin 10 Fakültesi, 2 Enstitüsü, Meslek Yüksekokulu, Milli Saraylar ve Tarihi Binalar Meslek Yüksekokulu, Yabancı Diller Yüksekokulu ve 25.000'den fazla öğrencisi bulunmaktadır.

İstanbul Devlet Mühendislik ve Mimarlık Akademisi ve bağlı mühendislik okulları ile Kocaeli Devlet Mühendislik ve Mimarlık Akademisi'nin ilgili fakülte ve bölümleri ve Kocaeli Meslek Yüksekokulu, 20 Haziran 1982 tarih ve 41 sayılı kanun hükmünde kararname ve bu kararnameyi değişikliklerle kabul eden 30 Mart 1982 tarih ve 2809 sayılı kanun ile birleştirilerek Yıldız Üniversitesi'ni oluşturmuştur.

Yeni üniversite, Fen-Edebiyat ve Mühendislik bölümlerini, Kocaeli'deki Meslek Yüksekokulunu, bir Fen Bilimleri Enstitüsü, bir Sosyal Bilimler Enstitüsü ve Rektörlüğe bağlı Yabancı Diller, Atatürk İlkeleri ve İnkılap Tarihi, Türk Dili, Beden Eğitimi ve Güzel Sanatlar bölümlerini bünyesine katmıştır.

Yıldız Teknik Üniversitesi, tarihi 1911'e dayanan Türkiye'nin en eski 3. üniversitesi olmasının yanı sıra İstanbul'da bulunan yedi devlet üniversitesinden biridir. Aynı zamanda ülkenin en iyi üniversitelerinden biri olarak kabul edilmektedir.

BORA İLCİ
KAAN EMRE KARA

İÇİNDEKİLER

ŞEKİL LİSTESİ	iv
ÖZET	v
ABSTRACT	vi
1 GİRİŞ	1
2 SİSTEM ANALİZİ VE FİZİBİLİTE	5
2.1 Sistem Analizi	5
2.2 Gereksinim Analizi	7
2.3 Fizibilite	8
2.3.1 Teknik Fizibilite	8
2.3.2 Yasal Fizibilite	8
2.3.3 Ekonomik Fizibilite	8
2.3.4 İşgücü ve Zaman Fizibilitesi	9
3 SİSTEM TASARIMI	10
3.1 Arayüz Tasarımı	10
3.2 Veri Tabanı Tasarımı	11
3.3 Nesneye Yönelik Tasarım	12
4 UYGULAMA SONUÇLARI ve TESTLER	14
4.1 Uygulama Tanıtımı	14
4.2 Test Sonuçları	14
4.3 Doğrulama	14
5 SONUÇ VE TARTIŞMA	15
Referanslar	16
Özgeçmiş	16

ŞEKİL LİSTESİ

Şekil 2.1	Bilgi üretim hattının üst düzey görünümü.	5
Şekil 2.2	İşgücü Fizibilitesi – Rol Bazlı Gantt Şeması	9
Şekil 3.1	Kullanıcı, arayüz ve ajanlar arasındaki etkileşim akışı.	11
Şekil 3.2	Dosya tabanlı veri bileşenleri ve ajan katmanı ile ilişkileri.	12
Şekil 3.3	Ajan sınıflarının basitleştirilmiş hiyerarşik görünümü.	13
Şekil 3.4	Kullanıcı, arayüz ve ajanlar arasındaki örnek etkileşim akışı.	13

**OTONOM AJAN YAKLAŞIMIYLA KOD VERİ
TABANINDAN EĞİTİM İÇERİĞİ ÜRETEY YAPAY ZEKA
TABANLI YAZILIMIN TASARIMI VE GELİŞTİRMESİ**

BORA İLCİ
KAAN EMRE KARA

Bilgisayar Mühendisliğı Bölümü
Bilgisayar Projesi

Danışman: Prof. Dr. Mehmet Sıddık AKTAŞ

BU BÖLÜM FİNAL RAPORUNDA GÜNCELLENECEK

Anahtar Kelimeler: BU BÖLÜM FİNAL RAPORUNDA GÜNCELLENECEK

ABSTRACT

DESIGN AND DEVELOPMENT OF AN ARTIFICIAL INTELLIGENCE-BASED SOFTWARE THAT GENERATES EDUCATIONAL CONTENT FROM A CODE DATABASE USING AN AUTONOMOUS AGENT APPROACH

BORA İLCİ
KAAN EMRE KARA

Department of Computer Engineering
Computer Project

Advisor: Prof. Dr. Mehmet Sıddık AKTAŞ

BU BÖLÜM FİNAL RAPORUNDA GÜNCELLENECEK

Keywords: BU BÖLÜM FİNAL RAPORUNDA GÜNCELLENECEK

1 GİRİŞ

Modern yapay zeka uygulamalarının temelinde, devasa metin ve kod veri kümeleri üzerinde eğitilmiş Büyük Dil Modelleri (LLM) yer almaktadır. Bu modeller, kendilerine sunulan bilgiyi anlama ve üretme konusunda yüksek kapasiteye sahip olsalar da, tek başlarına karmaşık ve çok aşamalı görevleri yerine getirmede sınırlı kalmaktadır. Bu noktada ajan tabanlı sistemler devreye girmektedir. Bir ajan, LLM'yi karar alma çekirdeği olarak kullanır; hedefe yönelik planlama yapar, dosya okuma veya kod yürütme gibi yardımcı araçları etkinleştirir ve çevreden aldığı geri bildirimle göre stratejisini günceller.

Örneğin finans teknolojileri alanında çalışan bir yazılım mühendisinin, on binlerce dosyadan oluşan yeni bir ödeme altyapısına katıldığını düşünelim. Geliştiricinin mimariyi anlaması, iş akışlarını kavraması ve basit bir değişikliği güvenle uygulayabilmesi çoğu zaman haftalar almaktadır. İnsan gözetiminde yürüyen bu uzun uyum sağlama süreci, kod tabanından yapılandırılmış bilgi çıkarabilen otonom sistemlerin yokluğunda daha da uzamakta; iş devri ve bakım maliyetleri yükselmektedir.

Bu mimarilerin en belirgin teknik kısıtlaması, LLM'lerin "bağlam penceresi" olarak bilinen veri işleme sınırıdır. Bir model aynı anda yalnızca sınırlı miktarda bilgiyi hafızasında tutabilir. Dolayısıyla kapsamlı bir yazılım projesinin tamamını tek seferde değerlendirmek geleneksel ajan yaklaşımlarıyla mümkün olamamaktadır.

Ajan tabanlı sistemler, yazılım geliştirme yaşam döngüsünün (SDLC) birçok aşamasını otomatize etme potansiyeline sahiptir. Bu sistemler, basit kod tamamlama araçlarının çok ötesine geçerek aktif roller üstlenmeye başlamıştır. Güncel uygulama alanları arasında şunlar bulunmaktadır:

- **Otomatik Hata Ayıklama (Debugging):** Bir kod tabanındaki hataları tespit etmek için test senaryoları yürütebilen, günlük (log) dosyalarını analiz edebilen ve hatanın kaynağını belirleyebilen ajanlar.

- **Talep Üzerine Kod Üretimi:** "Kullanıcılar için bir giriş (login) sayfası oluştur" gibi doğal dil taleplerini alıp, gerekli dosya yapılarını ve kod bloklarını üreten sistemler.
- **Mevcut Kodun Yeniden Yapılandırılması (Refactoring):** Verimsiz veya eski kod standartlarına göre yazılmış bölümleri analiz edip, daha performanslı veya modern eşdeğerleriyle güncelleyen ajanlar.
- **Kod Analizi ve Dokümantasyon:** Bu projenin de odak noktası olan, mevcut bir kod tabanını inceleyerek sistemin mimarisini, veri akışlarını ve bileşenlerin sorumluluklarını anlatan teknik dokümantasyon üreten sistemler.

Yazılım geliştirme süreçlerinde, "geniş ve yabancı bir kod tabanını anlamak, yazılım geliştiricilerin karşılaştığı en önemli zorluklardan biridir". Yeni bir geliştiricinin bir projeye dahil olması veya tecrübeli bir geliştiricinin projenin daha önce dokunmadığı bir bölümüne müdahale etmesi gerektiğinde, mevcut dokümantasyon genellikle eksik, güncel olmayan veya yüzeysel kalmaktadır. Bu durum, projelere adaptasyon sürecinin haftalar, hatta aylar sürmesine neden olarak ciddi bir zaman ve verimlilik kaybına yol açmaktadır.

Tıpkı geniş ölçekli çevrim içi platformlarda olduğu gibi, modern bir kurumsal kod tabanı binlerce dosya, on binlerce işlev ve milyonlarca satır kod içerebilir. Bu büyüklükteki ve yüksek derecede bağlı verinin, tek bir kişinin kısa sürede incelemesi ve anlamlandırması mümkün değildir. Son yıllarda bu sorunu aşmak için büyük dil modellerinden yararlanan ajan tabanlı yapay zekâ çözümleri geliştirilmektedir. Ancak söz konusu çözümler bile yüksek hacimli kod yığını bütünüyle kavramada bağlam penceresi sınırlamasına takılmaktadır.

Bu sınırlamanın temel nedeni, LLM'lerin belirli bir bağlam uzunluğunu aşamaması ve bu nedenle kod tabanındaki anlamsal bütünlüğü tek seferde kuramamasıdır. Kod tabanını bütünüyle modele yüklemek ya da aynı ajanın tüm dosyaları art arda okumasını beklemek, bağlamın dağılmasına ve yapılan değerlendirmelerin güvenilirliğinin azalmasına yol açmaktadır.

Bu çalışma kapsamında, bağlam penceresi kısıtlamasını aşmak için alt-ajan mimarisi adını verdiğimiz özgün bir yöntem benimsenmiştir. Söz konusu yaklaşımda kapsamlı analiz görevi daha küçük ve yönetilebilir alt görevlere bölünmektedir.

Projenin temel hedefi, söz konusu mimariyi kullanarak bir "yapay zekâ eğitmeni" prototipi geliştirmektir. Sistem iki ana aşamada işlemektedir:

1. **Aşama 1: Bilgi Bankası Üretimi:** Ana ajan, analiz edilecek yazılım projesini mantıksal modüllerine ayırır. Her bir bölümün analizi için, o bölüme özel, izole bir çalışma alanına ve kısıtlı araçlara sahip bir alt-ajan görevlendirir. Alt-ajan kendi analizini tamamlar ve bulgularını (markdown dosyaları, diyagramlar) belirlenen dizin altında merkezi bir bilgi bankası oluşturacak şekilde kaydeder. Bu yaklaşım, ana ajanın bağlamını temiz tutar ve görevi ölçeklenebilir hale getirir.
2. **Aşama 2: Eğitim Dokümanı Üretimi:** İlk aşamada üretilen ve kod tabanının bütününe özetleyen bilgi bankası bu bölümde girdi olarak kullanılır. Eğitim üretici bileşeni, söz konusu bilgi birikimini ve doğrudan kaynak koddan alınan güncel örnekleri bir araya getirerek geliştiricilere yönelik öğretici dokümanlar üretir.

Bu projenin seçilmesinin temel gerekçesi, özellikle üniversite öğrencileri ve stajyer geliştiriciler için kaynak hazırlanırken karşılaşılan bilgi derleme güçlüğüne ortadan kaldırmaktır. Klasik dokümantasyon süreçleri uzun zaman aldığından pek çok kurum güncel eğitim materyali sunamamaktadır; alt-ajan destekli yarı otonom bir çözüm bu boşluğu kapatmayı amaçlamaktadır.

Çalışmanın özgün katkıları aşağıdaki başlıklar altında toplanabilir:

- **Kararlı görev bölme yaklaşımı:** Ana ajanın kod tabanının dizin yapısını inceleyerek ilk seviye klasörlere göre tutarlı alt görevler oluşturması sağlanmış, böylece karmaşık planlama adımlarının yalın bir kural ile yürütülmesi mümkün kılınmıştır.
- **İzole çalışma alanları:** Her alt-ajanın yalnızca kendi çalışma dizinine yazabildiği, kod tabanına ise salt okunur eriştiği güvenli bir görev yürütme modeli tasarlanmış, bağlam taşması ve veri bütünlüğü riskleri azaltılmıştır.
- **İki aşamalı üretim hattı:** Bilgi bankası ile başlayan ve eğitim dokümanı üretimiyle tamamlanan süreç, dokümantasyon kalitesini yükseltirken ortaya çıkan içeriğin doğrudan öğretim amaçlı kullanılmasına olanak tanımaktadır.

Yazılım endüstrisinde geniş ve yabancı kod tabanlarını anlamak, dokümantasyon eksiklikleri nedeniyle ciddi bir darboğaz oluşturmaktadır. Yeni geliştiricilerin çok sayıda dosyayı tek tek incelemesi yüksek zaman maliyetine yol açmakta; yorumlama süreci ise kişinin bilgi düzeyine bağlı olarak tutarsız sonuçlar üretebilmektedir.

Bu alanda kullanılan ilk yöntemler, AutoGPT benzeri **tek ajanlı kod inceleme sistemleridir**. Bu sistemler araç kullanımıyla belirli görevleri yerine getirebilse de, **LLM bağlam penceresi sınırlaması** nedeniyle çok sayıda dosyayı aynı oturumda tutarlı biçimde analiz edememiş; uzun görevlerde **odak kaybı** ve hatalı çıkarımlar gözlenmiştir.

Sonraki aşamada ortaya çıkan **yönetici-işçi düzenindeki çoklu ajan mimarileri**, LangChain tabanlı örneklerde olduğu gibi görev dağılımı yaparak kısmi iyileştirme sağlamıştır. Ancak işçilerden gelen çıktının tekrar yöneticide birikmesi **bağlam taşmasına**, görevlerin yöneticinin belirlediği sıraya sıkı sıkıya bağlı olması ise **esneklik kaybına** neden olmuştur. Bu yapı yüksek hacimli depo analizlerinde ölçeklenebilirlik sorunlarını ortadan kaldıramamıştır.

Bu ön inceleme göstermektedir ki mevcut yaklaşımlar, özellikle:

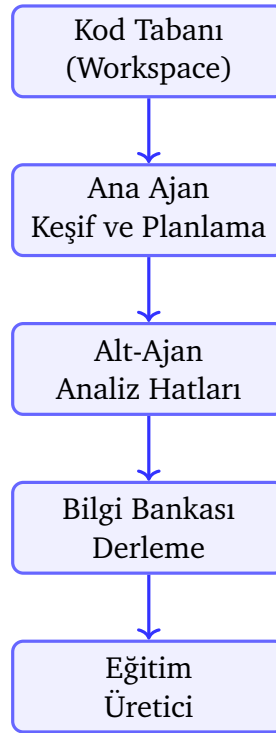
- **tek ajanlı sistemlerde bağlam sınırı ve uzun soluklu planlama güçlüğü,**
- **yönetici-işçi modellerinde verimlilik ve esneklik kısıtları,**
gibi temel problemleri çözmekte yetersizdir.

Dolayısıyla büyük kod tabanlarında tutarlı, izole ve ölçeklenebilir analiz yapabilmek için **alt-ajan mimarisine dayalı bir çözümün gerekliliği** ortaya çıkmaktadır. Her alt-ajanın kendi sorumlu dizininde çalışması, çıktılarının güvenli biçimde bir bilgi bankasında toplanması ve ana ajanın bağlam yükünden arındırılması bu yaklaşımın temel motivasyonunu oluşturur.

2 SİSTEM ANALİZİ VE FİZİBİLİTE

2.1 Sistem Analizi

Bu bölüm, Smolagents tabanlı alt ajan metodolojisini takip ederek “Otonom Ajan Yaklaşımıyla Kod Veritabanından Eğitim İçeriği Üreten” projenin kapsamını netleştirir. Amaç şekil 2.1’de gösterildiği üzere, kod tabanındaki bilgiyi çok ajanlı bir boru hattı ile derleyip kalıcı bir Bilgi Bankası üretmek ve bu çıktıyı pedagojik değeri yüksek eğitim materyallerine dönüştürmektir.

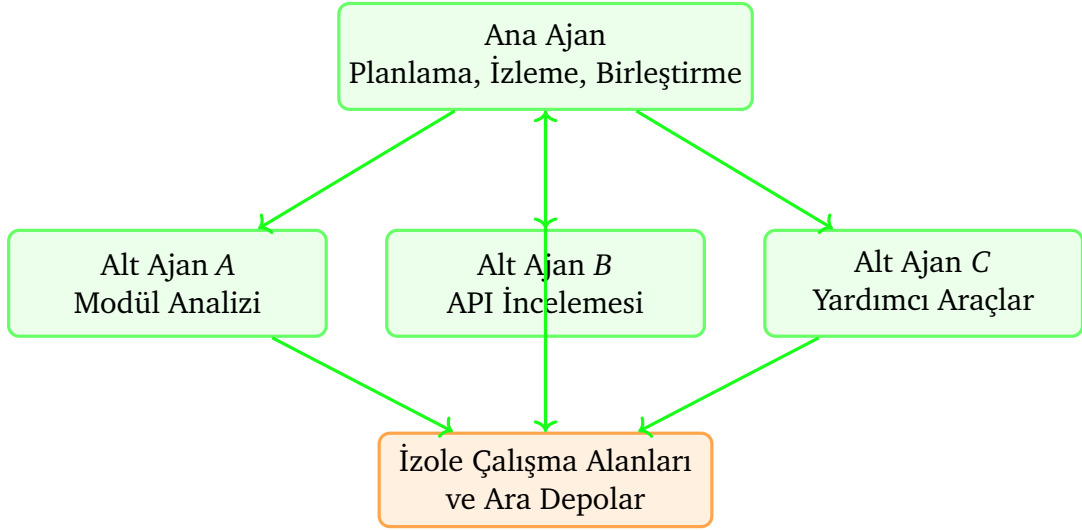


Şekil 2.1 Bilgi üretim hattının üst düzey görünümü.

Mimari, iki aşamalı Alt-Ajan kurgusu üzerine kuruludur. Ana Ajan keşif, planlama ve orkestrasyon görevlerini yürütür; bağlam yükünü azaltmak için işleri Alt-Ajanlara devreder. Alt ajanlar kendi çalışma alanlarında ayırık olarak analiz yapar, bulgularını kalıcı depolara kaydeder ve Ana Ajan’a raporlar. Ana Ajan bu çıktıları birleştirerek nihai Bilgi Bankası’nı oluşturur; ardından içerik üretimi için Eğitim Üretici bileşenini

tetikler.

Deterministik görev bölme yaklaşımıyla kaynak dosya altındaki ilk seviye klasörler ayrı alt ajanlara atanır. Alternatif olarak Ana Ajan, planlama parametreleri ve bir TODO listesi üzerinden görevleri dinamik biçimde güncelleyebilir. Alt ajanların kod tabanına yalnızca okuma, kendi çalışma alanlarına ise yazma yetkisi vardır; böylece bağlam yönetimi ve çıktı doğrulaması sadeleşir.



Başarı Kriterleri.

- **Kapsayıcı Tanıma:** Bilgi Bankası'nın kod tabanındaki temel bileşenleri doğru şekilde tanımlaması ve ilişkileri tutarlı biçimde çıkarması beklenmektedir. Bu kapsamda, yapısal kapsama oranı aşağıdaki ifade ile izlenebilir:

$$\text{Kapsam Oranı} = \frac{\text{Doğru Tanımlanan Bileşenler}}{\text{Toplam Bileşen Sayısı}}$$

Oranın %80–90 aralığında olması yeterli görülmektedir. Modüller arası ilişkilerde benzer biçimde en az %75–80 doğruluk hedeflenir.

- **Pedagojik Kalite:** Eğitim materyallerinin açıklayıcılığı, örneklerin niteliği ve görsel desteği genel bir bölüm kalitesi göstergesi ile değerlendirilebilir:

$$\text{Bölüm Kalitesi} = 0.5a + 0.3b + 0.2c$$

Burada a açıklama bütünlüğünü, b kod örneklerinin uygunluğunu, c ise diyagram ve görsel katkısını temsil eder. Ayrıca eğitim materyalinin öğrenmeye etkisi, basit bir ön test–son test farkı ile gözlemlenebilir:

$$\Delta = \text{Son Test} - \text{Ön Test}$$

Pozitif ve anlamlı bir Δ değeri, materyalin öğretici niteliğini destekler.

- **Süreç Dayanıklılığı:** Alt-ajan boru hattının kararlı çalışması sürecin başarısı için kritiktir. Görevlerin tamamlanma oranı şu şekilde izlenebilir:

$$\text{Tamamlanma Oranı} = \frac{\text{Başarılı Görevler}}{\text{Toplam Görev}}$$

Bu oranın yaklaşık %90 seviyesinde olması hedeflenir. Hata durumlarında sürecin toparlanabilmesi ise toparlanma oranı üzerinden değerlendirilir:

$$\text{Toparlanma Oranı} = \frac{\text{Toparlanabilen Hatalar}}{\text{Toplam Hata Sayısı}}$$

Son olarak, alt ajanların Ana Ajan üzerindeki bağlam yükünü azalttığı basit bir karşılaştırma ile gözlemlenebilir:

$$\text{Bağlam Azaltma Etkisi} = B_{\text{ana}} - B_{\text{alt}}$$

Pozitif bir fark, bağlam yönetiminde alt-ajan mimarisinin fayda sağladığını gösterir.

2.2 Gereksinim Analizi

Gereksinimler üç düzeyde sınıflandırılmıştır: kullanıcı rolleri, sistem modülleri ve işlevsel akış. Sistemin iki temel kullanıcısı bulunmaktadır. Birincisi, Bilgi Bankası ve eğitim çıktıları üzerinden kod tabanını öğrenen geliştiricidir. İkincisi ise ajan süreçlerini başlatan, izleyen ve üretilen çıktının bütünlüğünü doğrulayan sistem operatörüdür.

Sistem modülleri dört ana bileşenden oluşur. Ana Ajan; keşif, planlama, görev dağıtımı ve alt ajanlardan gelen çıktıları birleştirme görevlerini üstlenen merkezi unsurdur. Alt-Ajanlar; deterministik olarak veya koşula bağlı şekilde üretilen, belirli dizinleri veya görev kümelerini analiz eden ve izole çalışma alanlarında faaliyet gösteren ajanlardır. Bilgi Bankası deposu, tüm alt ajan özetlerinin toplandığı yapısal bir klasör düzeni sunarken, Eğitim Üretici bileşeni bu Bilgi Bankası'nı kullanarak pedagojik değer taşıyan eğitim materyallerini bağımsız bir çıktı klasörü altında üretir.

Sistemin işlevsel akışı ardışık beş aşamada gerçekleşir. İlk aşamada Ana Ajan kod tabanının dosya yapısını çıkarır, kritik bileşenleri analiz eder ve deterministik ya da dinamik olarak görev listesini oluşturur. İkinci aşamada bu görevler ilgili alt ajanlara dağıtılır. Üçüncü aşamada alt ajanlar, sorumlu oldukları kod parçalarını inceleyerek özetlerini kendi çalışma dizinlerine kaydeder. Dördüncü aşamada

Ana Ajan bu alt çıktıları bir araya getirerek bütünleşik Bilgi Bankası'nı oluşturur ve kalite kontrolünü gerçekleştirir. Son aşamada Eğitim Üretici, oluşturulan bu Bilgi Bankası'nı geliştiricilerin kolay öğrenebileceği bir pedagojik formata dönüştürür.

İşlevsel olmayan gereksinimler ise sistemin güvenlik, izlenebilirlik, genişletilebilirlik ve performans özelliklerini belirlemektedir. Sistem, özellikle *path traversal* saldırılarına karşı güvenli olacak şekilde tasarlanmalı; tüm API anahtarları gizli tutulmalı ve kod tabanına gömülmemelidir. Ayrıca ajanların karar verme süreçleri, özellikle görev delegasyonu ve dosya işlemleri Phoenix aracı üzerinden ayrıntılı biçimde loglanmalıdır. Mimari, yeni araçların veya ajan türlerinin eklenmesine açık bir genişletilebilirlik sunmalıdır. Son olarak performans açısından, bir prototip olması nedeniyle süreç karmaşıklığını azaltmak amacıyla alt ajanların paralel değil, sıralı olarak çalıştırılması tercih edilmiştir.

2.3 Fizibilite

2.3.1 Teknik Fizibilite

2.3.1.1 Donanım

Model eğitimi hedeflenmediği için yüksek donanım yatırımı gerekmez. LLM çağrıları API tabanlıdır; standart bir geliştirici bilgisayarı yeterlidir. İş yükü alt ajanlara bölüldüğü için bellek baskısı artmaz, uzun dosya okumaları engellenir.

2.3.1.2 Yazılım

- **İşletim Sistemi:** Windows, macOS veya Linux.
- **Programlama Dili:** Python 3.10 ve üzeri.
- **Çerçeveler ve Araçlar:** smolagents (ajan orkestrasyonu), LiteLLM (çoklu LLM entegrasyonu), Phoenix izleme, Gradio tabanlı basit arayüz.

2.3.2 Yasal Fizibilite

Kullanılan kütüphaneler MIT veya Apache 2.0 benzeri açık kaynak lisanslarıyla dağıtılır. Analiz edilecek kod tabanının proje ekibine ait olması veya lisans uyumunun sağlanması yeterlidir.

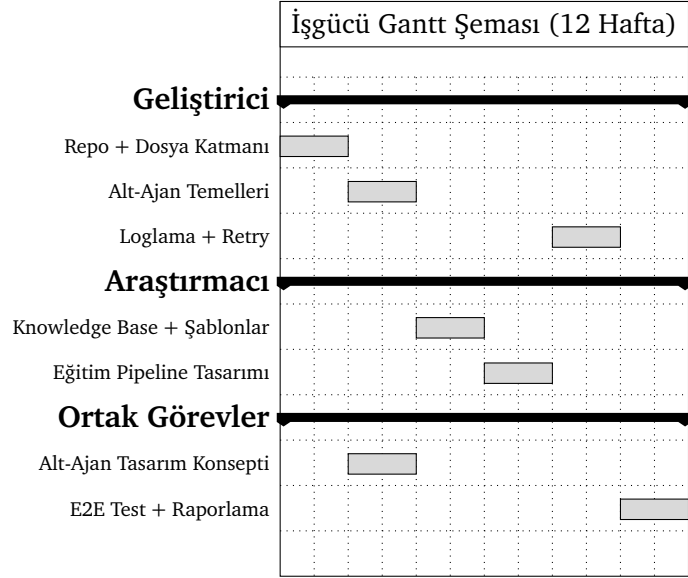
2.3.3 Ekonomik Fizibilite

Temel bileşenler ücretsizdir. Maliyet kalemleri çoğunlukla LLM API çağrıları ve gerekirse bulut barındırmadan oluşur. Geliştirme ve testte Google AI Studio veya OpenRouter üzerinden ücretsiz/düşük maliyetli modeller, son içerik üretiminde ise daha güçlü modeller tercih edilerek toplam gider düşürülebilir.

2.3.4 İşgücü ve Zaman Fizibilitesi

Ekip: 2 kişi (yazılım/araştırma odaklı).

Genel Yaklaşım: İlk sürüm için 8–12 hafta aralığında bir geliştirme süresi öngörülür. Detaylı şema şekil 2.2’de sunulmuştur.



Şekil 2.2 İşgücü Fizibilitesi – Rol Bazlı Gantt Şeması

Bu bölümde, “Smolagents Quickstart Template” tabanlı geliştirilen çok ajanlı yapay zekâ sisteminin tasarımı üç açıdan ele alınmaktadır: arayüz tasarımı, veri tabanı/veri yapısı tasarımı ve nesneye yönelik tasarım. Amaç, kod veritabanından eğitim odaklı içerik üretebilen bir sistemin kullanıcıyla etkileşimini, iç veri yapısını ve ajan temelli mimarisini tutarlı ve genişletilebilir bir bütün hâlinde sunmaktır.

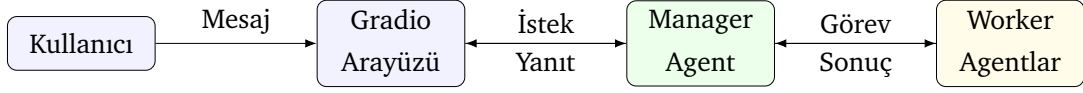
3.1 Arayüz Tasarımı

Sistem, kullanıcıyla etkileşimi sağlayan basit fakat genişletilebilir bir *chat* arayüzü üzerinden çalışmaktadır. Bu arayüz, Gradio tabanlı GradioAgentUI bileşeni kullanılarak tasarlanmış, kullanıcının doğal dilde mesaj gönderdiği ve çok ajanlı yapının cevabını gerçek zamanlı olarak görebildiği minimal bir yapıdadır.

Arayüz tasarımında öncelik, karmaşık kontrol öğeleri yerine sistemin temsil ettiği zekânın öne çıkarılmasıdır. Kullanıcı, tek bir metin alanına girdiği soruyu ya da isteği gönderir; bu girdi, doğrudan yönlendirici ajan olan *Manager Agent* tarafından işlenir. Yanıtlar, ajanların çalışma adımlarından bağımsız şekilde, sade bir sohbet geçmişi görünümünde sunulmaktadır. Böylece hem acemi hem de deneyimli geliştiriciler için kullanımı düşük bariyerli bir arayüz elde edilmiştir.

Arayüz katmanının temel işlevleri şu şekilde özetlenebilir: kullanıcı mesajını almak, bu mesajı yöneticiyi temsil eden ajana iletmek, alınan cevabı sohbet alanına yazmak ve gerektiğinde hata veya uyarı mesajlarını kullanıcıya yansıtmak. Arka planda çalışan izleme (*tracing*) ve günlükleme (*logging*) altyapısı, Phoenix aracı üzerinden ayrı bir arayüzde incelendiğinden, son kullanıcıya gösterilen ekran sade tutulmuştur.

Şekil 3.1’te kullanıcı ile ajanlar arasındaki etkileşimin üst düzey akışı gösterilmiştir.



Şekil 3.1 Kullanıcı, arayüz ve ajanlar arasındaki etkileşim akışı.

Bu tasarımda arayüz bileşeni olabildiğince ince tutulmuş, iş mantığı ve karar verme mekanizmaları bütünüyle ajan katmanına devredilmiştir. Böylece gelecekte farklı arayüzler (komut satırı, web paneli veya IDE eklentisi) eklenmek istendiğinde, yalnızca arayüz katmanının değiştirilmesi yeterli olacaktır.

3.2 Veri Tabanı Tasarımı

Proje bir veritabanı yönetim sistemi (VTYS) kullanmak yerine, ajanların ihtiyaç duyduğu kalıcı veriyi ve yapılandırma bilgilerini dosya tabanlı bir yapı üzerinden yönetmektedir. Böylece hem prototip aşamasında düşük karmaşıklık sağlanmakta hem de Smolagents tabanlı çok ajanlı yapının ihtiyaç duyduğu konfigürasyon verileri açık ve sade bir dizin yapısı ile dökümanite edilmektedir.

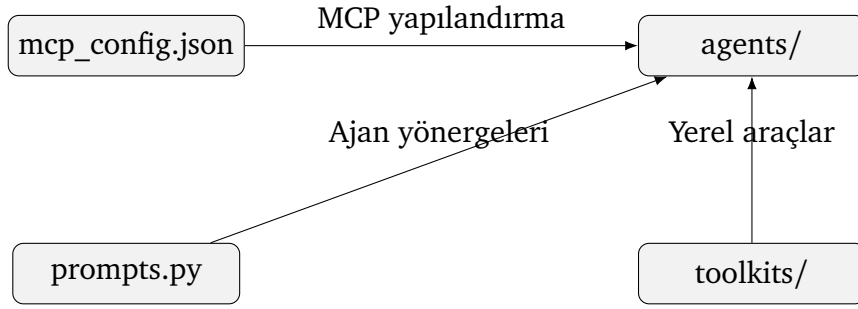
Sistemde üç tür kalıcı veri öne çıkmaktadır: (i) MCP istemci yapılandırmaları, (ii) ajan davranışlarını tanımlayan metinsel yönergeler, (iii) yerel araç (*toolkit*) tanımları.

3.2.0.0.1 MCP Konfigürasyon Deposu. `mcp_manager/mcp_config.json` dosyası, sisteme bağlı harici araç sağlayıcılarını tanımlar. Örneğin `deepwiki` adı verilen istemci, GitHub depolarına erişen bir MCP sunucusuna bağlanacak şekilde bu dosyada yapılandırılmaktadır. Manager Agent, sistem başlatılırken bu JSON dosyasını okuyarak hangi harici araçların mevcut olduğunu, hangi URL'ler üzerinden erişileceğini ve hangi aktarım yönteminin (*streamable-http* vb.) kullanılacağını belirler. Böylece metin ve kod tabanına ilişkin tüm harici bilgi kaynakları tek bir yapılandırma deposu üzerinden yönetilir.

3.2.0.0.2 Ajan Yönergeleri ve Şablonlar. `prompts/prompts.py` dosyası, her bir ajan için kullanılacak metinsel yönergeleri (*instruction prompt*) içerir. Manager Agent, araç çağıran worker ve kod üreten worker için farklı rol tanımlarına sahiptir; bu tanımlar, söz konusu dosyada fonksiyon ya da sabit dizgiler halinde toplanmıştır. Bu yaklaşım, ajan davranışını koddan bağımsız bir katmanda tutmaya olanak tanır ve gelecekte eğitim odaklı özel yönergelerin eklenmesini kolaylaştırır.

3.2.0.0.3 Yerel Araç Tanımları. `toolkits/` dizini altında, Smolagents'in `@tool` dekoratörü ile işaretlenmiş fonksiyonlar birer yerel araç olarak tanımlanmaktadır. Örneğin `example_joke_toolkit.py` dosyasında yer alan `get_joke` fonksiyonu, kod üreten worker ajan tarafından çağrılabilir ve sistemin davranışını test etmek için kullanılabilir. Aynı model, ileride kod tabanını analiz eden veya dosya sisteminden güvenli okuma yapan araçlar eklenirken de izlenecektir.

Bu yapı, klasik bir ilişkilisel veritabanı şeması yerine, dosya bazlı bir “veri tabanı tasarımı” sunar. Şekil 3.2’de temel dosya ve dizinlerin ilişkilendirilmiş görünümü verilmiştir.



Şekil 3.2 Dosya tabanlı veri bileşenleri ve ajan katmanı ile ilişkileri.

Bu sayede, sistemin kalıcı verisi tek bir veritabanına bağlı olmadan yönetilebilmekte; proje gereksinimleri değiştikçe yeni MCP istemcileri, yeni araç kümeleri veya yeni prompt dosyaları kolaylıkla eklenebilmektedir.

3.3 Nesneye Yönelik Tasarım

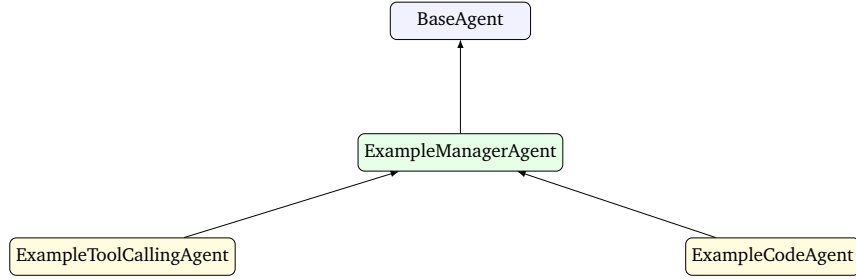
Uygulama, Smolagents çatısını temel alan nesneye yönelik bir mimariyle tasarlanmıştır. Her ajan, Python tarafında bir sınıf (veya sınıf sarmalayıcısı) olarak temsil edilmekte; böylece sorumlulukların açık biçimde ayrılması, kodun yeniden kullanılabilirliği ve genişletilebilirliği sağlanmaktadır.

Temel ajan hiyerarşisi aşağıdaki şekilde özetlenebilir:

- **BaseAgent:** Ortak davranışları ve arayüzü tanımlayan soyut katman.
- **ExampleManagerAgent:** Kullanıcıdan gelen istekleri alan, görevleri worker ajanlara dağıtan ve sonuçları birleştiren yönlendirici ajan.
- **ExampleToolCallingAgent:** Harici MCP araçlarını (örneğin DeepWiki) kullanarak depo ve kod tabanı hakkında bilgi toplayan worker ajan.
- **ExampleCodeAgent:** Yerel toolkits (örneğin şaka üretme aracı) üzerinden kod üreten veya yardımcı işlemler gerçekleştiren worker ajan.

Bu yapı, tek sorumluluk ilkesi (SRP) doğrultusunda her sınıfın belirli bir görev alanına odaklanmasını sağlar. Manager ajan, planlama ve orkestrasyon sorumluluğunu üstlenirken; worker ajanlar analiz, araç çağırma veya kod üretme gibi daha dar kapsamlı işleri yürütür.

Şekil 3.3’de ajan hiyerarşisinin sadeleştirilmiş bir sınıf diyagramı gösterilmektedir.

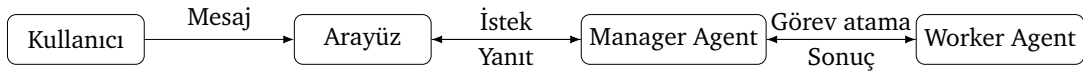


Şekil 3.3 Ajan sınıflarının basitleştirilmiş hiyerarşik görünümü.

Ajanlar arası etkileşim, özellikle kullanıcıdan gelen bir mesajın sistem içinde hangi adımlardan geçtiğini göstermek açısından önemlidir. Bu çalışmada, tipik bir etkileşimde şu adımlar izlenmektedir:

1. Kullanıcı, Gradio arayüzü üzerinden doğal dilde bir mesaj gönderir.
2. Arayüz bu mesajı ExampleManagerAgent örneğine iletir.
3. Manager ajan, mesajın niteliğine göre uygun worker ajanı (veya ajanları) seçer; harici bilgi gerektiğinde ExampleToolCallingAgent, yerel kod veya yardımcı işlem gerektiğinde ExampleCodeAgent devreye girer.
4. Worker ajan, kendisine atanmış araçları (toolkits veya MCP istemcileri) kullanarak görevi yerine getirir ve sonucu manager ajana döner.
5. Manager ajan, gerektiğinde birden fazla worker çıktısını birleştirir ve son cevabı arayüze iletir.

Bu etkileşim, Şekil 3.4’de basitleştirilmiş bir sıra diyagramı olarak gösterilmiştir.



Şekil 3.4 Kullanıcı, arayüz ve ajanlar arasındaki örnek etkileşim akışı.

Bu nesneye yönelik tasarım sayesinde sistem, yeni worker ajanlar, yeni toolkit’ler veya farklı türde MCP istemcileri eklenmek istendiğinde minimum değişikliklerle genişletilebilecek esnek bir yapıya kavuşmuştur. Aynı zamanda, sorumlulukların net bir biçimde ayrılması, hata ayıklama ve izleme sürecini kolaylaştırarak projenin bakım maliyetini azaltmaktadır.

4 UYGULAMA SONUÇLARI ve TESTLER

- 4.1 Uygulama Tanıtımı
- 4.2 Test Sonuçları
- 4.3 Doğrulama

5 SONUÇ VE TARTIŞMA

BİRİNCİ ÜYE

İsim-Soyisim: BORA İLCİ

Doğum Tarihi ve Yeri: 04.10.2003, Bursa

E-mail: bora.ilci@std.yildiz.edu.tr

Telefon: 0541 200 31 04

Staj Tecrübeleri: Orion Innovation Türkiye Ar-Ge Departmanı

İKİNCİ ÜYE

İsim-Soyisim: KAAN EMRE KARA

Doğum Tarihi ve Yeri: 24.09.2002, İstanbul

E-mail: emre.kara@std.yildiz.edu.tr

Telefon: 0530 776 8079

Staj Tecrübeleri: Nef Solution Yazılım Departmanı

Proje Sistem Bilgileri

Sistem ve Yazılım: Windows/macOS/Linux, Python 3.10+

Gerekli RAM: 8 GB

Gerekli Disk: 1 GB